

Documento técnico CI/CD DevOps

15 AGOSTO 2026

Universidad de la sabana

Creado por:

Juan Sebastian Buitrago Romero

Jhonatan Winston Sotelo Santofimio

Carlos Arturo Salazar Castañeda

Santiago Eduardo Munoz Castillo



Documentación Técnica – Pipeline CI/CD con GitHub Actions y Jenkins

Introducción

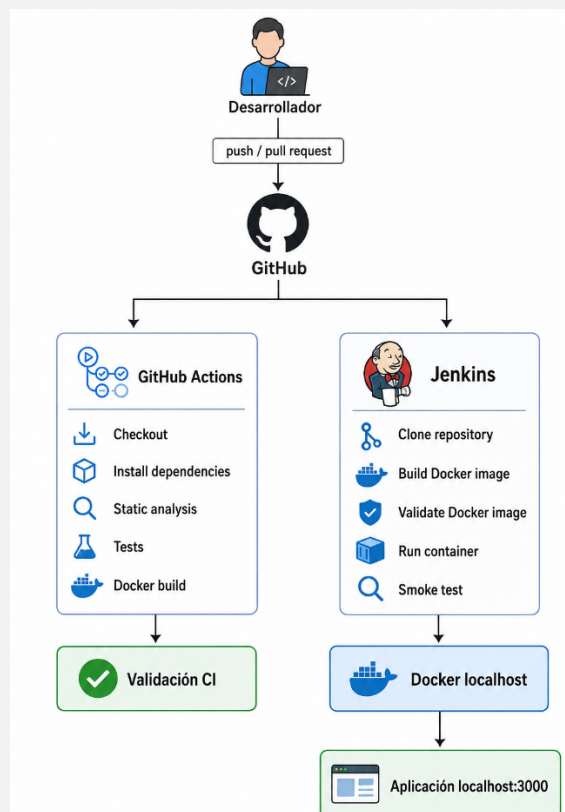
El objetivo de este laboratorio es implementar un flujo básico de Integración Continua (CI) y Entrega Continua (CD) para una aplicación web almacenada en un repositorio GitHub.

Para ello se utilizan dos herramientas:

- GitHub Actions, encargado del proceso de Integración Continua.
- Jenkins, encargado de definir y ejecutar el proceso de construcción y validación del artefacto Docker.

La aplicación utilizada en el laboratorio está desarrollada con Node.js y Express y dispone de pruebas automatizadas mediante Jest y análisis estático mediante ESLint.

La solución implementada sigue el siguiente flujo



1. Aplicación utilizada

Para realizar las pruebas del pipeline se ha creado una aplicación web sencilla utilizando Node.js y Express.

La aplicación expone principalmente dos endpoints:

- GET /
- GET /health

El endpoint /health permite comprobar automáticamente si la aplicación se encuentra funcionando correctamente.

Una respuesta correcta es:

```
{  
  "status": "UP"  
}
```

Este endpoint resulta especialmente útil para realizar el Smoke Test desde Jenkins después de arrancar el contenedor Docker.

2. Integración Continua con GitHub Actions

GitHub Actions se utiliza como herramienta de Continuous Integration (CI).

Su responsabilidad consiste en validar automáticamente cualquier modificación realizada sobre el código antes de que pueda continuar hacia las siguientes fases del ciclo de entrega.

La configuración se encuentra en:

`.github/workflows/ci.yml`

3. Activación del workflow

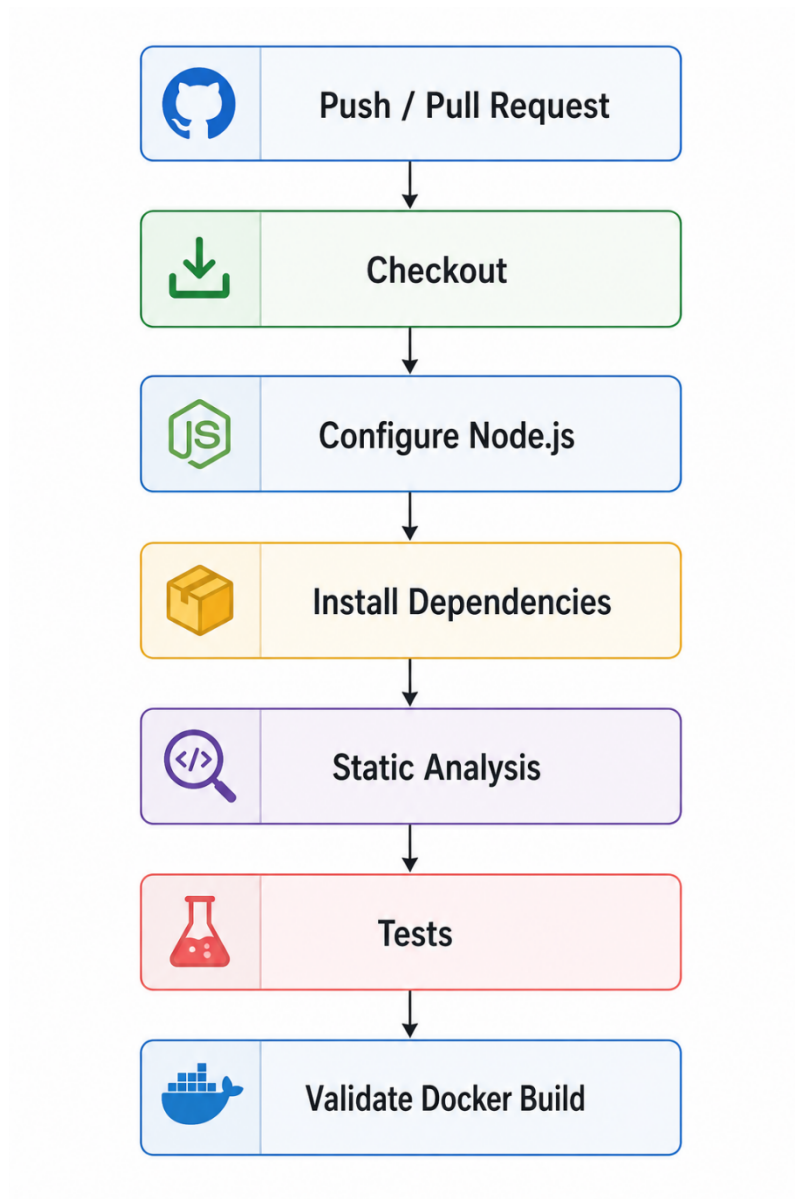
El pipeline se configura para ejecutarse automáticamente cuando se produce un:

- push
un:
- pull_request

sobre las ramas:

- main
- develop

Esto permite que cualquier cambio enviado al repositorio sea validado automáticamente.
El flujo CI implementado es:



4. Steps de GitHub Actions

4.1. Checkout source code

El primer step obtiene el código fuente del repositorio GitHub.

- name: Checkout source code
uses: actions/checkout@v4

La acción actions/checkout descarga el contenido del repositorio dentro del runner temporal utilizado por GitHub Actions.

Este paso es necesario porque los siguientes comandos necesitan acceder al código fuente, al package.json, al Dockerfile y a las pruebas.

4.2. Configure Node.js

El siguiente step configura Node.js dentro del runner:

```
- name: Configure Node.js
  uses: actions/setup-node@v4
  with:
    node-version: "20"
```

En este laboratorio se utiliza Node.js 20.

Este step garantiza que el entorno utilizado por GitHub Actions dispone de una versión conocida y reproducible de Node.js.

4.3. Install dependencies

Una vez configurado Node.js se instalan las dependencias de la aplicación:

```
- name: Install dependencies
  run: npm install
```

El comando consulta el archivo:

- package.json

e instala las dependencias necesarias.

Entre ellas se encuentran:

- Express.
- Jest.
- Supertest.
- ESLint.

Esta fase prepara el entorno necesario para poder ejecutar posteriormente el análisis estático y las pruebas.

4.4. Static code analysis

El siguiente step realiza un análisis estático del código mediante ESLint:

```
- name: Static code analysis
  run: npm run lint
```

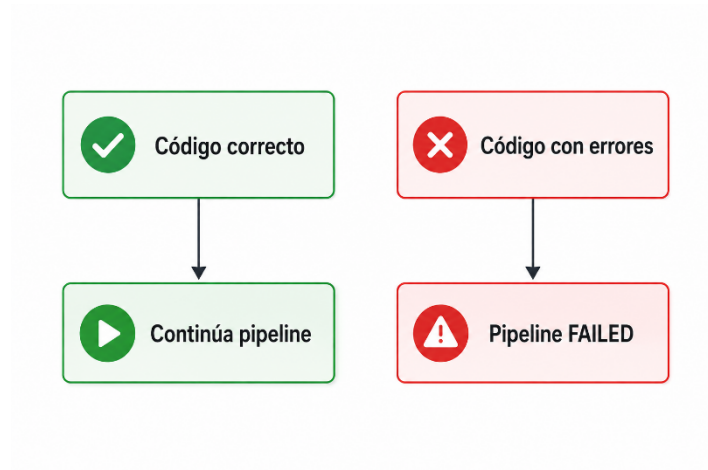
Este comando ejecuta el script definido en **package.json**.

El objetivo del análisis estático es detectar problemas como:

- Variables no definidas.
- Variables declaradas pero no utilizadas.
- Errores básicos de calidad del código.

- Incumplimientos de las reglas configuradas en ESLint.

Si ESLint encuentra un error configurado como bloqueante, el comando devuelve un código de salida diferente de cero y GitHub Actions detiene el pipeline.



5. Ejecución de pruebas

El siguiente step ejecuta las pruebas automatizadas:

- name: Run tests
run: npm test

Para este laboratorio se utiliza Jest como framework de pruebas y Supertest para realizar peticiones HTTP sobre la aplicación Express.

Entre otras comprobaciones, las pruebas verifican que:
GET /

devuelve un código HTTP:

- 200

y que:

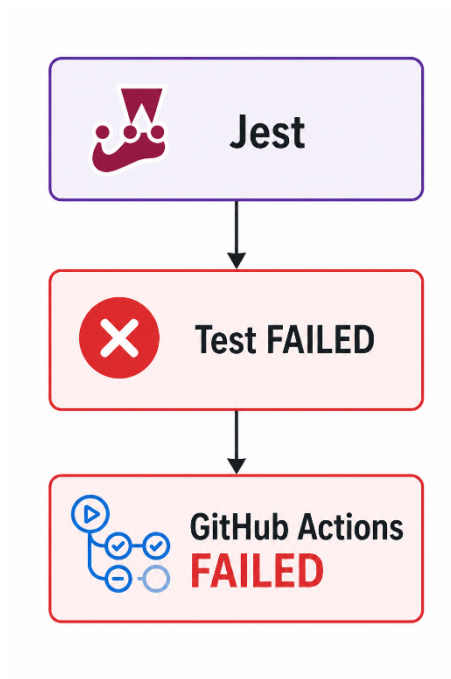
GET /health

devuelve:

```
{  
  "status": "UP"  
}
```

Este step permite comprobar que los cambios realizados en el código no han roto el comportamiento básico de la aplicación.

Si una prueba falla:



De esta manera se evita continuar con una versión del código que no supera las pruebas definidas.

6. Validación de la imagen Docker

El último step del proceso CI valida que la aplicación pueda empaquetarse correctamente como una imagen Docker.

- name: Validate Docker build
run: docker build -t cicc-lab-webapp:\${{ github.sha }} .

Docker utiliza el archivo:

- Dockerfile

Para construir la imagen.

Como tag se utiliza el identificador del commit:

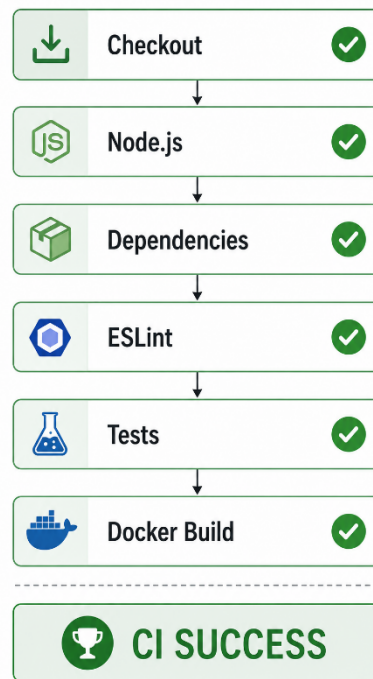
- github.sha

Conceptualmente se obtiene una imagen como:

- cicc-lab-webapp:a821df...

Este paso es importante porque comprueba que el código no solamente supera las pruebas, sino que también puede convertirse correctamente en un artefacto desplegable.

El workflow de GitHub Actions termina entonces con:



7. Pipeline de Jenkins

Jenkins se utiliza para implementar la segunda parte del laboratorio.

La definición del pipeline se almacena dentro del repositorio mediante el archivo:

- Jenkinsfile

Esto permite aplicar el concepto de Pipeline as Code, ya que la definición del proceso de automatización se versiona junto con el código fuente.

En este laboratorio Jenkins se encuentra instalado directamente sobre Windows y utiliza el Docker Engine disponible en la máquina local.

El flujo implementado es:



8. Definición de parámetros y entorno

Antes de ejecutar los stages, el pipeline define los valores dinámicos y globales que se utilizarán en todo el proceso.

Se reciben como parámetros:

- GIT_REPOSITORY: La URL del código fuente en nuestro caso es: <https://github.com/juansebastian-br/CI-CD-with-jenkins.git>
- GIT_BRANCH: La rama a construir en nuestro caso es: main
- IMAGE_NAME: El nombre local de la imagen Docker en nuestro caso es: cicd-lab-webapp.

Se configuran como variables de entorno fijas:

- CONTAINER_NAME: cicd-lab-webapp-test
- APP_PORT: 3000

Esto permite reutilizar el mismo Jenkinsfile y modificar valores clave sin alterar el código fuente del pipeline.

.

Después de este stage, Jenkins dispone dentro de su workspace de:

- src/
- test/
- Dockerfile
- package.json
- Jenkinsfile
- .github/

9. Stage – Clone repository

El primer stage de Jenkins obtiene el código fuente desde GitHub utilizando los parámetros definidos y ejecutando el siguiente comando:

- git branch: "\${params.GIT_BRANCH}", url: "\${params.GIT_REPOSITORY}"

Después de este stage, Jenkins dispone dentro de su workspace de todos los archivos necesarios para la construcción

10. Stage – Check Docker

Antes de iniciar la construcción, este stage verifica que el entorno de Jenkins tenga Docker correctamente instalado y en ejecución.

Se ejecutan los siguientes comandos de validación:

- **whoami**: Para identificar el usuario que está ejecutando el proceso.

- **docker --version:** Para comprobar la versión instalada.
- **docker ps -a:** Para listar los contenedores existentes en el sistema local.

11. Stage - Build Docker image

Este stage construye la imagen Docker de la aplicación. Primero, Jenkins utiliza un bloque script para generar dinámicamente variables basadas en el número de ejecución.

Por ejemplo, para el build número 15:

- **IMAGE_TAG:** 15
- **FULL_IMAGE:** cicd-lab-webapp:15

El comando ejecutado es:

- `docker build -t cicd-lab-webapp:15 -t cicd-lab-webapp:latest .`

Esto genera dos referencias: el tag específico del build y un tag general a la última versión construida. La imagen permanece almacenada en el Docker Engine local.

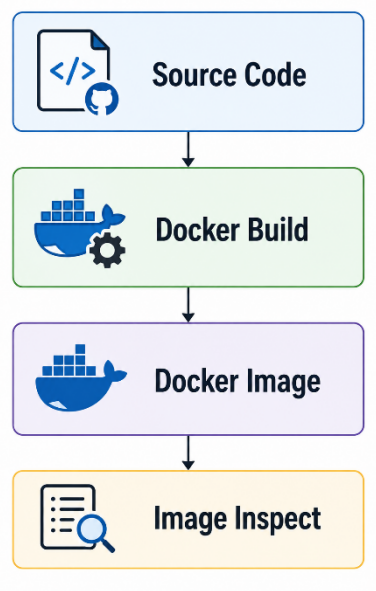
12. Stage – Validate Docker image

Después de construir la imagen, se comprueba que Docker la haya generado correctamente.

Se utiliza:

- `docker image inspect cicd-lab-webapp:15`

Este stage permite detectar inmediatamente un problema y detener el pipeline si la imagen esperada no se construyó de manera exitosa.



13. Stage – Remove previous container

Antes de arrancar el nuevo contenedor, se elimina cualquier versión anterior utilizada por el laboratorio para evitar conflictos de nombres o puertos ocupados.

El comando ejecutado es:

- `docker rm -f cicd-lab-webapp-test`

Al agregar el sufijo `2>/dev/null || true`, se le indica al pipeline que ignore los errores en caso de que el contenedor no exista previamente, permitiendo que el flujo continúe sin interrupciones.

14. Stage – Run container

Una vez limpio el entorno, Jenkins ejecuta un contenedor utilizando la imagen recién construida.

El comando ejecutado es conceptualmente:

- `docker run -d --name cicd-lab-webapp-test -p 3000:3000 cicd-lab-webapp:15`

El parámetro `-d` ejecuta el contenedor en segundo plano. El parámetro `--name` asigna un nombre conocido al contenedor basándose en las variables de entorno. El parámetro `-p` publica el puerto 3000 del contenedor sobre el puerto 3000 del host.

Como nosotros hicimos la implementación de este laboratorio en un servidor en la nube, a aplicación queda temporalmente disponible en:

<http://3.144.124.190:3000>

15. Stage – Check container

Este stage se encarga de validar el estado del contenedor recién levantado para asegurarse de que no haya fallado inmediatamente tras el arranque.

Se utilizan los siguientes comandos:

- `docker ps -a --filter "name=cicd-lab-webapp-test"` Para ver su estado de ejecución.
- `docker logs cicd-lab-webapp-test` Para capturar los registros de arranque de la aplicación.

16. Stage – Smoke Test

Una vez iniciado el contenedor se realiza una comprobación funcional mínima o Smoke Test.

Se ejecuta:

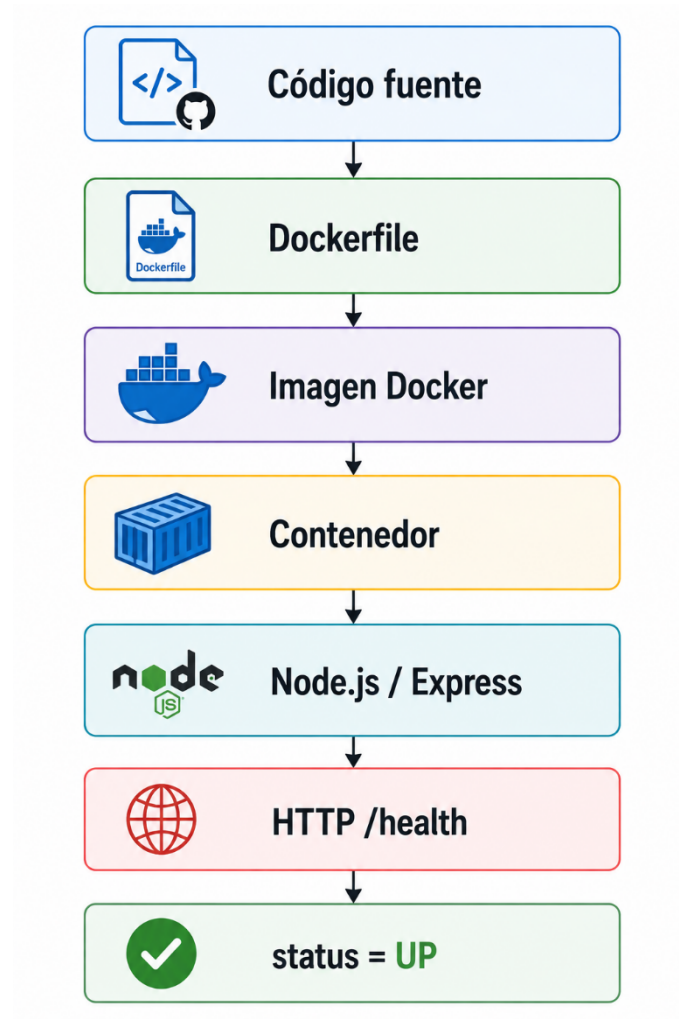
- `curl --fail http://localhost:3000/health`

El objetivo es verificar que no solamente existe la imagen y que el contenedor se ha iniciado, sino que la aplicación realmente responde mediante HTTP.

La respuesta esperada es:

```
{
  "status": "UP"
}
```

Por tanto, este stage valida la cadena completa:



Si curl no puede conectarse o recibe una respuesta HTTP de error, el comando devuelve un código de salida de error y Jenkins marca el stage como fallido.

17. Post Actions de Jenkins

El pipeline dispone también de acciones posteriores a la ejecución. Independientemente del resultado, se elimina el contenedor temporal: `cicd-lab-webapp-test`

Esto evita dejar contenedores ejecutándose después de finalizar el laboratorio. La imagen Docker, sin embargo, permanece disponible localmente. Puede comprobarse mediante:
`docker images`

En caso de ejecución correcta Jenkins informa de que el pipeline ha terminado satisfactoriamente. Si alguno de los stages falla, Jenkins marca la ejecución como:

-
- FAILED

Permitiendo revisar los logs del stage correspondiente.

18. Configuración de Docker en Jenkins sobre linux

Para hacer la configuración desde cero dentro del servidor de AWS lo primero que tuvimos que hacer fue hacer el proceso de instalación de docker en el servidor, para esto seguimos los siguientes pasos:

- Ejecutar el comando de instalación de docker: **sudo yum install -y docker**
- Después de la instalación, debemos iniciar la herramienta: **sudo systemctl start docker**
- Luego configuramos el docker para que siempre se inicie cuando el sistema operativo arranque: **sudo systemctl enable docker**
- Finalmente validamos que el servicio docker este funcionando: **sudo systemctl status docker**

Tras finalizar la instalación de docker lo que tenemos que hacer es configurar los permisos para que el usuario que ejecuta Jenkins pueda usar el dominio de docker, para eso hacemos lo siguiente:

- Ejecutamos el comando **sudo usermod -aG docker jenkins** para dar los permisos al usuario
- Luego reiniciamos el servicio de Jenkins para aplicar los cambios: **sudo systemctl restart jenkins**

Después de seguir estos pasos ya pudimos hacer la creación del Job de Jenkins para poder correr el pipeline y desplegar nuestra aplicación

19. Separación de responsabilidades entre GitHub Actions y Jenkins

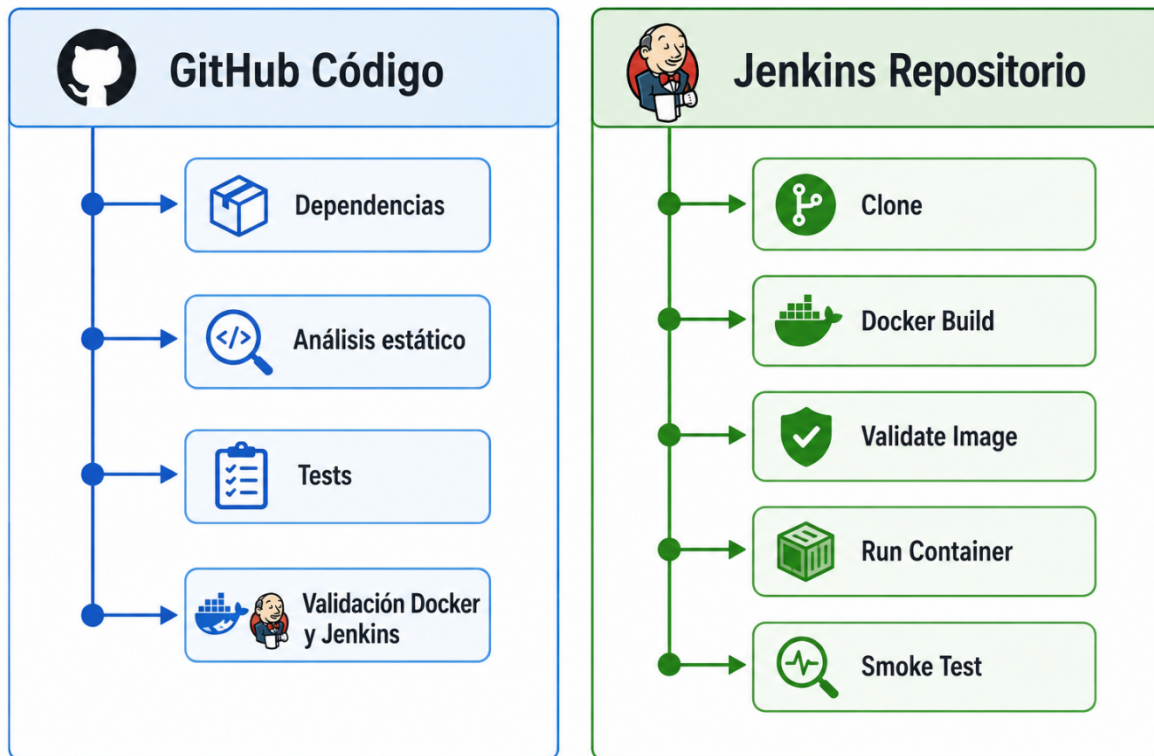
La solución divide las responsabilidades del proceso CI/CD.

GitHub Actions – CI

GitHub Actions se ocupa principalmente de validar el código

Jenkins – CD

Jenkins se ocupa de generar y comprobar el artefacto ejecutable



Su objetivo en el laboratorio es demostrar que la versión puede empaquetarse y ejecutarse correctamente.